CHAPTER 5

Memory Is the Product

My AskAI's Customer Support Architecture

Anusha Prakash - 002306070

In February 2024, a passenger named Jake Moffatt contacted Air Canada's AI support agent to ask about bereavement fares following a family death. The agent told him he could purchase a full-price ticket and apply for the discounted bereavement rate retroactively within 90 days, a policy Air Canada explicitly stated did not exist. Moffatt followed the agent's instructions. Air Canada refused the refund. The British Columbia Civil Resolution Tribunal ruled against Air Canada, holding the company legally responsible for its agent's output. Air Canada was ordered to pay $812.02 in damages and legal fees.

The agent that failed Moffatt was not running a faulty model. It had been trained on Air Canada's product and pricing context. It responded fluently and coherently to a natural-language question about a real company policy. What it did not have was a mechanism for grounding its response in a verified, retrievable source before generating it. It produced policy language from its weights the learned, statistical residue of its training data, rather than from a document it had retrieved and could cite. When the policy it generated did not match the policy that existed, there was no architectural check to detect or prevent the discrepancy.

This is not a model failure. It is an architecture failure. And it has a name: the absent Tier 5 citation constraint. Understanding exactly what that means at the level of retrieval mechanisms, vector stores, and prompt assembly is what this chapter is about.

> The question this chapter answers: What is the minimum memory architecture that would have prevented the Air Canada failure and what breaks, specifically and observably, when any component of it is absent?

The Intuition That Is Wrong

Before building the correct architecture, it is worth spending a moment with the wrong one because the wrong one is intuitive, widely deployed, and initially appears to work.

The intuition runs like this: large language models are powerful because they can reason over long contexts. The more information in the prompt, the better the answers. If Air Canada's full policy documentation had simply been loaded into every prompt pasted in alongside the conversation, the agent would have had access to the correct bereavement fare policy. It would have retrieved the right answer from within the context window. No separate retrieval architecture required. Just a bigger prompt.

This intuition is plausible enough that it is how most teams build their first AI support agent. They load the full documentation corpus into the system prompt, add the conversation history, send it to the model, and observe that early tests look good. The agent seems to know everything. The team ships it.

Then, at scale, something strange happens. Response quality does not hold. The agent begins missing references that are clearly in the prompt. It contradicts itself within a single conversation. It resolves tickets correctly for simple queries but fails on moderate-length ones, despite having every relevant document in front of it. And sometimes as in the Air Canada case it generates confident, fluent responses that have no basis in any document at all.

What happened is described precisely in Liu et al. (2023). In a controlled experiment, models were given prompts containing a relevant document placed at different positions within a context window ranging from 1,000 to 32,000 tokens. When the relevant content appeared at the beginning or end of the window, accuracy was high. When it appeared in the middle, accuracy dropped by as much as 20 percentage points not because the model could not read it, but because transformer attention degrades for tokens that are neither near the beginning nor near the end of the sequence.

This is not a quirk of one model. It is a consequence of how attention is computed. The attention score A between a query vector Q and key vector K in scaled dot-product attention is:

$$A(Q, K) = \text{softmax}(QK^T / \sqrt{d_k})$$

At the token level, this computation is symmetric — every token attends to every other token within the window. But models trained on autoregressive sequences develop positional biases through their training data: documents tend to open with their most important content, conversations tend to close with their most recent and urgent information. The middle of a long prompt where a documentation corpus loaded wholesale would place most of its content is precisely the position the model has been least rewarded for attending to during training.

The result, called the Lost in the Middle effect, means that the 'bigger context = better' intuition inverts at scale. Beyond roughly 4,000–8,000 tokens of effective context, adding more content to a single flat prompt actively degrades the model's ability to retrieve and use that content. The Air Canada agent's bereavement policy document, buried in the middle of a long prompt, would have been less reliably attended to than a document retrieved separately and injected at a controlled position.

But the Air Canada failure is not even primarily a positional bias failure. It is worse than that: the documentation may not have been loaded at all. The agent generated policy content from its weights pattern-matched from training data rather than from any retrieved document. This is the failure that a citation-constrained retrieval architecture prevents: the agent is prohibited from generating policy-relevant content unless it can cite a source chunk it retrieved. No retrieval, no response. If the knowledge base does not contain the answer, the agent escalates rather than invents.

Here is the translation from mechanism to consequence the chain this chapter requires the student to hold together. At the token level: attention weight degrades for content in the middle of a flat context window. At the session level: this produces unreliable retrieval the agent misses relevant content proportional to its distance from the prompt boundaries, and sometimes generates plausible content with

no retrieved grounding at all. At the business level: unreliable retrieval converts a customer interaction into a legal liability. The $812.02 Air Canada judgment is a business outcome whose proximate cause is a missing citation constraint in the retrieval architecture. The architecture is the leverage point.

What This Argument Does and Does Not Cover

The central claim of this chapter is that AI support agent failures root-cause to memory architecture, not model capability is correct for the class of failures described here. It is not a universal claim.

A genuine model failure looks like this: the retrieval pipeline returns the correct document, the document is injected at the correct position in the prompt, the context window contains all and only the relevant information and the model still produces a wrong answer. This happens. Models hallucinate in the presence of full grounding. Models exhibit bias in output generation that is not reducible to missing context. Models fail on specific reasoning patterns that are not retrieval problems.

The diagnostic sequence for a production failure should proceed in this order:

1. Check architecture first: Is the right content in the prompt, at the right position, with no context overflow? If no, the failure is architectural.
2. Check retrieval second: Did the vector search return the correct chunk? Was hybrid search active for the query type? Did the 24-hour sync produce a stale result? If no, the failure is retrieval-layer.
3. Check the prompt assembly third: Is the retrieved content positioned correctly not buried in the middle of a long context? Is the system prompt injecting conflicting instructions? If no, the failure is prompt assembly.
4. Suspect the model last: Only after architecture, retrieval, and prompt assembly are verified correct should the model itself be examined as the failure source.

This diagnostic order matters because model replacement is expensive, slow, and often ineffective when the actual failure is architectural. The Air Canada failure was diagnosed as a model problem the agent 'made things up' but the correct fix was architectural: enforce retrieval-before-generation as a hard constraint, require citation for any policy claim, and escalate when no grounding source is available. No model change was required. The architecture was the leverage point.

> Scope boundary: The five-tier memory architecture described in this chapter is designed for reactive support agents systems that respond to inbound queries. It is not the architecture for proactive autonomous agents, long-running background tasks, or multi-agent coordination systems. Those architectures require different memory primitives, different write policies, and different failure mode taxonomies. Later chapters address those cases explicitly.

The Architectural Tetrahedron: An Analytical Instrument

Every design choice described in this chapter can be evaluated through four lenses that together constitute the Architectural Tetrahedron: Structure (the physical arrangement of data and components), Logic (the reasoning mechanism that connects structure to behavior), Implementation (the specific tools and configurations used), and Outcome (the resulting system behavior and its consequences). These four lenses are not a labeling exercise. They are a generative tool they tell the architect what questions to ask before any code is written.

The Tetrahedron has a known failure mode as an analytical instrument: when two of its lenses pull in opposite directions, it does not resolve the conflict automatically. A Structure choice that produces excellent Outcome metrics (fast retrieval, low cost) may require a Logic that is brittle or unauditable. An Implementation that is operationally simple may produce Outcomes that require human oversight the team cannot afford. The Tetrahedron surfaces these tensions it does not adjudicate them. That adjudication is the human architect's job.

Applying the Tetrahedron to Air Canada

Before applying the Tetrahedron to My AskAI's architecture where all four answers are provided in the surrounding text apply it to Air Canada's. The tribunal record establishes the following facts about the agent's architecture: it was trained on Air Canada's product context; it generated policy language in response to a natural-language question; the generated content was not sourced from a retrieved document; there was no citation requirement that would have flagged the absence of grounding. Use the Tetrahedron to reconstruct what the architecture must have looked like.

Tetrahedron Element	Reconstructed from Tribunal Record	The Missing Element
Structure	LLM with training-time product context. No evidence of a separate, quarriable knowledge base for policy retrieval.	No vector store or document index for policy grounding
Logic	Generate a response to the user's policy question using training-time knowledge. No constraint requiring retrieval before generation.	No retrieval-before-response logic; no citation requirement
Implementation	Conversational AI agent deployed on Air Canada's support channel. No hybrid search, no top-k document injection, no chunk citation in output.	No RAG pipeline; no Tier 5 archival memory layer
Outcome	Agent fabricated a non-existent policy with high fluency and confidence. Company held legally liable. $812.02 in damages.	Legal liability as the measurable Outcome of absent retrieval architecture

Table 1. Tetrahedron applied to Air Canada. Reconstruct the missing elements before reading Table 2.

The Tetrahedron analysis of Air Canada produces the chapter's core architectural claim as a conclusion, not an axiom: the absence of a retrieval-before-generation constraint at the Logic level, which requires a knowledge store at the Structure level and a RAG pipeline at the Implementation level is the direct architectural cause of the Outcome. The model's fluency is not the problem; it is what makes the problem costly. A less fluent model would have produced an obviously wrong answer. A fluent model with no retrieval constraint produced a legally actionable one.

Applying the Tetrahedron to My AskAI

Table 2 maps the same four lenses to My AskAI's production architecture. The student should complete this table independently before reading forward the answers are derivable from the Air Canada analysis above and the tier descriptions that follow.

Tetrahedron Element	My AskAI (initial mapping — revise after failure modes section)	Defect (add after Table 3)
Structure	Five memory tiers: context window (Tier 1), vector DB keyed by user ID (Tier 2), knowledge graph (Tier 3), DAG rule engine (Tier 4), document vector store (Tier 5)	
Logic	Tiered retrieval routes only relevant content to the model's attention window; citation constraint requires a retrieved source chunk for any policy claim; human-gated write path for Tier 3	
Implementation	Qdrant Cloud for Tiers 2 and 5; Redis for Tier 1; hybrid semantic + lexical search; 24-hour sync cycle; Improve feature for Tier 3 writes	
Outcome	75%+ ticket deflection at ~$0.10/resolved conversation; source citations in every response; escalation when no grounding source available	

Table 2. Tetrahedron for My AskAI. Complete the Defect column after reading the Tier 5 defect subsection and the failure modes section.

The Five-Tier Memory Stack

To overcome the inherent statelessness of LLMs, architects must provide an external cognitive stack. This stack has five distinct tiers, each optimized for a specific access pattern, data type, and latency requirement. What makes the stack architectural rather than additive is that each tier is scoped: it stores only what it needs to store, retrieves only what it needs to retrieve, and writes only through a path

appropriate to its data's sensitivity and reversibility. The tiers do not substitute for each other. Removing any one produces a failure that the others cannot compensate for.

Tier 1: Working Memory - The Hot Layer

Working memory is the active conversation window: the most recent turns, the current query, and any ephemeral variables the agent is tracking mid-task. Its scope is a single session. Its retrieval mechanism is direct injection no lookup required. Its failure mode is the Amnesiac Pivot, described in the failure modes section.

The critical design constraint is the Context Window Tax. Naive scaled dot-product attention grows as $O(n^2)$ in memory with sequence length n. Flash Attention reduces this to $O(n)$ in memory, but the token budget constraint remains real: every unnecessary token is a cost and a potential source of positional bias degradation. Efficient Tier 1 management requires two mechanisms: a sliding window retaining only the most recent k turns in raw form, and a summarization pass that compresses older turns into a structured summary injected at the beginning of the context window the position with the highest reliable attention weight.

Tier 2: Episodic Memory - The Experience Layer

Episodic memory stores timestamped records of past interactions: who contacted support, what issue they reported, when, how it was resolved, and whether the resolution held. This is the tier that enables recurrence detection the ability to recognize that a customer is returning with a problem that was previously marked resolved.

In implementation, episodic memory uses a vector database (Qdrant Cloud in My AskAI's production system) where past interactions are encoded as dense embeddings. New queries retrieve the top-k most similar past episodes for that user. The critical design constraint is scope isolation: episodic records are keyed by user ID. A query for one user must never return episodes from another. Scope collapse at Tier 2 is a privacy failure, not a performance failure.

My AskAI exposes Tier 2 in two forms. In the standard tier, episodic memory is lightweight: an AI-generated structured summary passed to the human agent at escalation handoff. In the enterprise tier, historic ticket data is embedded and indexed, providing full cross-session recall. This pricing structure creates tiered failure modes: organizations with the standard tier cannot detect recurrences; those with the enterprise tier can.

The Escalation Summary as a Pedagogical Tool

The AI-generated escalation summary, the structured handoff from the agent to the human support team is worth examining as a learning object, not just as a product feature. The scaffold handles: extracting the key issue, the steps already taken, and the customer's current state from the conversation history. What the student must still evaluate: whether the summary is complete (did it omit a critical detail that changes the escalation path?), whether it is accurate (did it represent the resolution status correctly?), and

whether its omissions would constitute a system failure if the human agent acted on it without reading the full transcript.

A student who accepts the escalation summary without auditing it has learned to trust automated summarization which is exactly the posture that produces the Air Canada failure. The appropriate exercise: given a conversation log with a deliberate omission in the AI-generated summary, identify what the summary missed, explain how that omission would affect the human agent's response, and specify the minimum fields that any escalation summary must contain to be operationally safe. That is what makes the summary a pedagogical instrument rather than a convenience feature.

Tier 3: Semantic and Relational Memory - The Truth Layer

Semantic memory stores abstracted facts and relationships that are true across interactions: product compatibility charts, organizational policies, the mapping between a user and their support plan tier. In production systems, this tier is often implemented using a knowledge graph (Neo4j is common) alongside relational database tables, enabling multi-hop reasoning: an agent can trace that User A belongs to Organization B, which has Premium Support Plan C, and route their ticket to priority handling automatically.

The write policy for Tier 3 is the most consequential architectural decision in this tier. In My AskAI, the semantic knowledge base is human-mediated for writes: the Improve feature requires a human support agent to review and approve any correction before it enters the knowledge base. The reason this matters and the reason it is not optional is illustrated by the next section's canonical failure case.

Tier 4: Procedural Memory - The Playbook Layer

Procedural memory encodes the agent's behavioral rules: when to invoke which tool, what the terminal conditions for a resolution workflow are, and at what thresholds the agent must escalate rather than resolve autonomously. In My AskAI, this is implemented as directed acyclic graphs (DAGs) defining escalation paths and tool usage protocols.

The Planner Loop failure mode - infinite retry cycles caused by underspecified terminal conditions is a Tier 4 failure. Every DAG branch must define a terminal state (resolved, escalated, timed_out, or failed_explicitly) and every tool call must have a timeout that transitions to the next terminal state if no response arrives within threshold.

Tier 5: Archival Memory - The Deep Layer

Archival memory is the company's documentation corpus: help articles, product guides, policies, and historical ticket logs. Content is retrieved via RAG the current query is converted to a vector embedding and compared against the indexed document corpus, with the top-k most similar chunks injected into the prompt at a controlled position. Every AI response must include a citation to the source chunk that grounded its answer. When no grounding source is available, the agent escalates rather than generates.

This citation constraint is what would have prevented the Air Canada failure. An agent prohibited from generating policy claims without a retrieved source chunk cannot produce a plausible-but-fabricated policy. It can only say 'I found no policy matching this query' and escalate. The legal liability of a confident wrong answer is replaced by the operational cost of an escalation a cost that is both smaller and recoverable.

Where Tier 5 Itself Fails: Internal Defects

Presenting Qdrant and RAG as a solved solution would leave the student with an incomplete model. The archival tier has its own inherent failure modes that must be designed against.

First, vector similarity degrades for short and precise queries. Embedding models are trained predominantly on sentence-length and paragraph-length text. A two-word query like 'bereavement policy' produces a sparse embedding that matches many documents weakly rather than one document strongly. The fix which My AskAI implements is hybrid search: blending semantic similarity with BM25 lexical matching for queries that contain precise identifiers, policy names, or error codes.

Second, the 24-hour sync cycle creates a consistency window. During the period between a documentation update and the next sync, the knowledge base is stale. An agent asked about a policy that changed this morning will retrieve yesterday's version and answer with outdated information confidently, with a citation. The citation makes the error harder to catch, not easier, because it appears authoritative. Production systems requiring sub-day freshness must implement event-driven sync rather than scheduled sync.

Third, chunking strategy determines retrieval precision in ways that are not visible to the end user. A document chunked at 512 tokens may split a policy procedure across two chunks, returning the first (which describes eligibility) without the second (which describes the application process). The agent then answers half the question correctly and omits the operative step. Chunk size, overlap, and boundary detection are active design decisions with measurable precision consequences.

Tier	Name	Storage	Access Speed	Failure If Absent	Internal Defect
Tier 1	Working Memory	Context window	Immediate	Amnesiac Pivot: goal token eviction	Context overflow; Lost in the Middle for long sessions
Tier 2	Episodic Memory	Vector DB, user-keyed	Fast (~10 ms)	Episodic Blind Spot: recurrence invisible	Scope collapse if user ID isolation fails
Tier 3	Semantic Memory	Knowledge graph + relational DB	Medium (~50 ms)	No org-level context; incorrect routing	Schema drift if ontology is not maintained; unchecked write corruption

Tier 4	Procedural Memory	DAG rule engine	Compiled / instant	Planner Loop: infinite retry	Stale DAGs if procedures change without redeployment
Tier 5	Archival Memory	Vector DB + document store	Retrieval (~100–500 ms)	Fabricated output; no citation; Air Canada failure mode	24-hr stale window; poor precision for short queries; chunking boundary splits

Table 3. Five memory tiers with absence failures and internal defects. Complete the Defect column in Table 2 using this table before proceeding.

Case Study: My AskAI's Architecture

The Qdrant Pivot: Why RAG Replaced Fine-Tuning

My AskAI began as a general-purpose 'chat with your data' tool. The founders pivoted to customer support after observing that the failure mode of the general tool confident, fluent, ungrounded responses was particularly costly in a support context, where wrong answers have direct business and legal consequences. The architectural pivot that made the product viable was replacing fine-tuned models with a RAG-based retrieval system backed by Qdrant Cloud.

Fine-tuned models are brittle for support contexts: they bake product knowledge into weights at training time. When documentation changes as it does continuously in any active product the model must be retrained. An agent trained on last month's refund policy will answer confidently and incorrectly about this month's. RAG externalizes the knowledge into a retrievable, updatable store. The model's weights remain static; the knowledge it draws on does not. This architectural choice directly addresses the Air Canada failure mode: the knowledge is not in the weights, it is in the store, and the store can be verified, audited, and corrected without retraining.

The discovery that semantic search alone was insufficient for precise identifiers led to the hybrid search implementation described in the Tier 5 defects subsection. The architecture is continuously refined by its own failure modes which is why the 'Where Tier 5 Itself Fails' section is not an appendix but a design specification.

Why the Deflection Rate Differential Is an Architectural Outcome

Table 4 shows My AskAI achieving 75–82% deflection against Zendesk AI's reported 23–66%. The temptation is to treat this as a product claim and move on. A student who does that has missed the chapter's core lesson. The deflection rate differential is an architectural outcome, and the mechanism is derivable.

A support agent with a flat-context architecture (no Tier 2 episodic store) cannot detect recurrences. Every returning customer with an unresolved issue is treated as a new ticket. The agent re-resolves using the same procedure. The resolution fails again. The ticket re-opens. Deflection rate is the fraction of contacts resolved without human escalation and each re-opened ticket that could have been escalated on the first return contact instead consumes an additional agent interaction and then escalates anyway. The flat-context agent's deflection rate is depressed not because it resolves fewer issues but because it fails to identify the cases where escalation on the first contact would have produced a better outcome than a second identical resolution attempt.

Tiered retrieval improves deflection by enabling a class of resolutions that are not possible without episodic memory: the recurrence identification, the acknowledgment that the prior resolution failed, and the decision to escalate immediately rather than re-resolve. This is a qualitative architectural difference in capability, not a quantitative tuning difference. Zendesk's architecture, as documented in the benchmark period, did not include cross-session episodic recall at the standard tier. That architectural absence is the proximate cause of the deflection rate differential.

Note: All figures in Table 4 are self-reported by My AskAI as of early 2026. The mechanism argument above is independently derivable from architecture; the specific numbers should be treated as estimates pending independent verification.

Feature	My AskAI	Zendesk AI	Intercom Fin	eesel AI
Architecture	Tiered RAG (Qdrant)	Native / Integrated	Linear RAG	Linear RAG
Starting Price	$199/month	Enterprise only	$0.99/resolution	$299/month
Deflection Rate (self-reported)	75%–82%	23%–66%	~60%	~81%
Cost per Ticket (est.)	~$0.10	~$0.85	$0.99	~$0.25
Setup Time	< 15 minutes	Weeks / months	Moderate	< 15 minutes
Memory Persistence	Cross-session (enterprise)	Limited	Session-based	Session-based

Table 4. Competitive landscape, early 2026. Self-reported figures. Deflection rate differential mechanism is derived in the preceding section.

Failure Modes: When Architecture Fails the Agent

Every failure mode in this section is architecturally caused the model responds correctly to the information it has. The architecture has failed to provide the right information at the right time, or has permitted an action the architecture should have constrained.

Failure 1: The Amnesiac Pivot (Tier 1 Failure)

The Amnesiac Pivot occurs when goal-defining tokens are displaced from the working memory buffer mid-conversation. Here is the causal chain at the token level:

5. A user initiates a refund request. The system prompt and first two turns establish 'Refund Request' as the active goal approximately 200 tokens at the beginning of the context window.
6. The agent retrieves the Refund Policy from Tier 5 and begins the resolution procedure.
7. The user asks three clarifying questions about a different product, adding approximately 600 tokens to the context.
8. The context window reaches its configured max_history_turns limit. The FIFO buffer evicts the oldest turns to stay within budget. The 'Refund Request' context the oldest content is evicted first.
9. The agent answers the final clarifying question, scans working memory for open tasks, finds none, and closes the session without completing the refund.

Trigger: set max_history_turns = 3 in the demo notebook, initiate a refund, then send three unrelated questions. The agent closes the session without completing the refund reproducibly, across model variants. Fix: goal pinning inject a structured 'active goal' record into the system prompt at every turn, at the beginning of the context window where attention weight is highest. Goal pinning adds approximately 50 tokens per turn, a cost measurably smaller than the token cost of a re-opened ticket.

Failure 2: The Episodic Blind Spot (Tier 2 Failure)

The fictional analog of this failure is straightforward: a customer contacts support with a recurring problem, the agent re-resolves it identically to the previous session, the resolution fails again. The customer churns. This chapter's worked exercise uses this pattern call it the Priya scenario because it is clean, controllable, and observable in the demo notebook.

But the failure mode has a real-world analog that is worth naming: any customer support operation running without cross-session episodic memory is systematically unable to identify chronically unresolved issues. The customers who contact support three or more times with the same problem are, statistically, the customers most likely to churn and most likely to generate negative public commentary. The episodic blind spot does not cause visible errors it produces invisible recurrences that accumulate silently into churn and reputation damage. The failure mode is not an exception; it is the baseline behavior of any stateless support system.

Trigger (Exercise B): run a support session, clear the episodic store, run an identical second session, compare responses. The second session produces no recurrence flag. Restore the store, rerun: the agent flags the recurrence and escalates immediately.

Failure 3: Context Rot (Tier 1 + Tier 5 Interaction Failure)

Context Rot occurs when the total injected context exceeds the model's effective attention range. The result is not an error, the result is silent degradation. The model continues to respond fluently, but it begins preferring content at the prompt boundaries over content in the middle. Citations are missed. The agent contradicts itself within a single response. The responses are fluent and wrong.

At the mechanism level: the softmax-normalized attention weight assigned to a token in the middle of a 32,000-token context is, on average, lower than tokens near the boundary because the gradient signal during training was weaker for middle positions. This is not a bug; it is a consequence of training on data where document beginnings and endings are disproportionately information dense. The architectural defense is context budgeting: cap total injected content below the effective attention range, summarize Tier 1 rather than growing it without bound, and limit Tier 5 retrieval to top 3 chunks rather than top-10.

Failure 4: The Planner Loop (Tier 4 Failure)

When procedural memory lacks explicit terminal conditions, the agent enters an infinite retry cycle. The agent calls a resolution tool, receives an ambiguous response, evaluates it against underspecified success criteria, cannot determine whether the condition is met, and re-invokes the tool. In production systems at scale, this consumes tokens and burns through API credit limits without resolving anything. The fix is DAG hygiene: every branch must define a terminal state, and every tool call must have a timeout that transitions to the next terminal state if no response arrives within threshold.

Failure Mode	Memory Tier	Root Cause	Observable Symptom	Architectural Fix
Amnesiac Pivot	Tier 1	FIFO eviction of goal tokens from context window	Agent closes session without completing original task	Goal pinning at system prompt start; sliding window with summarization
Episodic Blind Spot	Tier 2	Absent cross-session store; every query treated as first occurrence	Recurrence indistinguishable from first report; chronic issues accumulate silently into churn	User-keyed episodic vector store; enterprise historic ticket indexing
Context Rot	Tier 1 + 5	Total injected context exceeds effective attention range; positional bias on middle tokens	Missing citations, self-contradiction, fluent but inaccurate responses	Context budget cap; summarize Tier 1; limit Tier 5 to top 3 chunks
Planner Loop	Tier 4	Underspecified terminal conditions in procedural DAGs	Infinite retries; token drain; no resolution	Explicit terminal states in all DAG branches; tool call timeouts

Table 5. Failure modes by tier. All four are architecturally caused; all four are demonstrable in the exercise notebook.

> Tetrahedron revision checkpoint: Return to Table 2. Add the Defect column using Table 3 as your reference. Then add a fifth row: 'Air Canada.' Fill in all four Tetrahedron elements for Air Canada's architecture using Table 1. The fifth row should make visible which element of My AskAI's architecture was specifically designed to prevent the Air Canada failure mode.

The Mandatory Human Decision Node: The Samsung Failure and the Write Boundary

In April 2023, Samsung engineers used ChatGPT for three separate work tasks: debugging semiconductor equipment source code, optimizing battery code, and summarizing a confidential internal meeting. In each case, the proprietary content entered OpenAI's servers as part of the conversation context. Under OpenAI's terms of service at the time, user inputs could be used to improve the model. The proprietary source code and meeting transcripts permanently exited Samsung's organizational boundary. Samsung subsequently banned internal use of external AI tools and began developing an internal alternative.

The Samsung incident is not a model failure. The model worked as designed. It processed the input and produced useful output. The failure was architectural: there was no write boundary between the engineer's working session (Tier 1 - the active context) and an external system's persistent storage. The engineer's working memory became the training data of a third-party model. No policy violation, no warning, no human decision node. The write happened automatically, by default.

This is the Deferred Decision Hazard: a system that defers the decision about what to persist until after the data has already been written. The decision is not made; it is avoided. And the cost of avoiding it arrives later, at a moment when reversal is impossible.

The Decision Node in the Demo Notebook

In the demo notebook, an AI scaffold handles the initial memory tier enumeration. For a given helpdesk description, it proposes:

- Tier 1 buffer: 10 most recent turns
- Tier 2 store: vector index over all raw past customer interaction transcripts
- Tier 3 map: relationship between Product SKU and Version Number

Before this architecture is instantiated, the scaffold halts at the Mandatory Human Decision Node. The Samsung incident is the reason this node exists.

```
# MANDATORY HUMAN DECISION NODE
# ──────────────────────────────────────────────────────────────────────
# The scaffold proposes embedding raw chat transcripts into Tier 2.
# Before proceeding verify this assumption holds for your
# compliance profile.
#
# The agent cannot make this decision. It requires knowledge of:
#   - Your organization's legal deletion obligations (GDPR Art. 17)
#   - Whether PII in vector embeddings can be reliably purged
#   - Your acceptable risk posture for a Deferred Decision Hazard
#   - Whether the write to Tier 2 is reversible without full
```

```
#    index reconstruction (it is not, in most vector store configs)
#
# Samsung engineers made this decision by not making it.
# Document your verification or rejection below:
# ────────────────────────────────────────────────────────────────────────────

HUMAN_DECISION = {
   'accepted': False,
   'reason': 'Raw transcripts create a Deferred Decision Hazard.
            PII embedded in a vector store is practically
            un-deletable without full index reconstruction.
            Samsung case: no write gate = permanent data exit.',
   'modification': 'Insert SanitizationNode(redact_pii=True)
                between Tier 1 capture and Tier 2 write.'
}
```

The decision to reject the raw transcript write and insert a sanitization node is not a technical optimization. It is a judgment about irreversibility. The sanitization node adds 50–100 ms of processing latency per session close. The un-scrubbed transcript store adds regulatory liability that cannot be quantified in advance and, as Samsung demonstrates, cannot be reversed after the fact. That trade-off 100 ms of latency versus permanent IP exposure, is the human decision. The agent cannot make it because the input to the decision is the organization's legal posture and risk tolerance, neither of which is in any training corpus or knowledge base the agent can retrieve.

Operational Decision Nodes: Not Just at Design Time

The PII write boundary is a design-time decision node it governs what the architecture permits before the system goes live. But a production support agent also requires mandatory human decision nodes during operation. Designing a system with sound architecture but no operational oversight nodes produce a system that is architecturally correct and operationally ungoverned.

Three operational nodes are mandatory in any production deployment of this architecture:

Operational Node	Trigger Condition	Who Decides	Authority
Resolution Confidence Threshold	Agent's internal confidence score drops below defined threshold (e.g., 0.70) for a resolution. Agent is prohibited from autonomously closing the ticket.	Human support agent reviews and either approves the resolution or escalates	Override any agent close decision; reopen any auto-closed ticket within 24 hours
Tier 3 Write Gate	A human agent uses the Improve feature to propose a correction to the knowledge base. Write is staged, not immediate.	Knowledge base owner reviews the proposed correction for accuracy, scope, and potential side	Reject any proposed write; approve with modifications; trigger rollback if an approved write produces

		effects before approving the write	measurable deflection rate degradation
Deflection Rate Audit Trigger	Deflection rate drops more than 10 percentage points over any 7-day rolling window	System administrator or support lead reviews conversation logs from the period, identifies failure mode category (Tier 1, 2, 4, or 5), and determines whether architectural intervention or knowledge base update is required	Halt the deployment; revert to prior knowledge base snapshot; escalate to architecture review

Table 6. Mandatory operational decision nodes. Architecture is the design constraint; these nodes are the operational governance layer.

The Tier 3 Write Gate is the operational equivalent of the design-time PII decision node it prevents unchecked writes from corrupting the semantic memory layer. The deflection rate audit trigger is the operational equivalent of the Air Canada fix: it catches the systemic effect of architectural degradation before it accumulates into legal liability. Neither of these nodes is automatable. Both require a human who has authority to halt the system.

LLM as Scaffolding: What the Agent Handles and What You Must

The AI scaffold in the demo notebook handles the enumeration of memory tier candidates the mechanical task of mapping a system description to a proposed five-tier structure. This is the cognitive load it reduces: taxonomy application, a task that is well-defined and pattern-matchable. The cognitive capacity freed is directed toward the design judgments the scaffold cannot perform: evaluating the proposed structure against security requirements, compliance obligations, and production constraints that are invisible to the model.

What the Scaffold Handles	What the Student Must Do
Enumerate memory tier candidates for a given system description	Evaluate each tier against the organization's security and compliance requirements
Propose initial write policies for each tier	Decide whether proposed write paths create irreversible or high-liability data states (cf. Samsung)
Estimate token costs and latency for proposed configurations	Weigh token cost against recall quality given the use case's specific query distribution
Generate DAG terminal state options for Tier 4 procedures	Verify that terminal states map to actual downstream system capabilities and SLA requirements

Propose Tier 5 chunking strategy and chunk size	Evaluate whether proposed chunk boundaries would split critical procedures test against the four provided edge cases
Generate escalation summary from conversation history	Audit the summary for completeness and accuracy; specify the minimum fields a safe escalation summary must contain; identify what a missing field would cost the receiving human agent

Table 7. Scaffold handles taxonomy; student handles judgment. The right column is where architectural thinking lives.

Three additional scaffold applications are available in the demo notebook that the chapter's exercises do not fully exploit:

- LLM-assisted DAG terminal state generation: given a procedure description, the scaffold proposes terminal states. The student audits for completeness and tests against three provided edge cases where the proposed states would produce incorrect transitions.
- LLM-assisted chunk boundary evaluation: given a document, the scaffold proposes a chunking strategy. The student evaluates whether the proposed boundaries would split any critical policy procedure across chunks, and proposes one boundary revision with a justification.
- LLM-assisted incident diagnosis: given a conversation log with a failure, the scaffold identifies which memory tier is most likely absent. The student verifies by running the targeted notebook exercise and either confirms or challenges the scaffold's diagnosis with evidence from the conversation log.

What the Architecture Cannot Do

The five-tier architecture has known capabilities and known limits. The student who understands both is more dangerous than the student who only knows the happy path.

- It cannot learn from its own mistakes without human intervention. The machine cannot write to Tier 3 or Tier 5 directly. Errors persist until a human approves a correction through the Improve feature. This is not a missing feature it is the design choice that prevents silent knowledge corruption. The alternative, a self-writing knowledge base, can incorporate its own errors without any visible signal that the knowledge base is degraded.
- It cannot detect recurrences without the enterprise tier. Standard-tier organizations cannot identify the Episodic Blind Spot failure mode. This means the architecture's most important capability for chronically unresolved issues is gated behind a pricing tier.
- It cannot guarantee sub-day freshness. The 24-hour sync cycle produces a consistency window. An agent citing an article updated this morning will cite yesterday's version. The citation makes the error harder to catch.
- It cannot maintain context across very long sessions without Tier 1 summarization. Sessions exceeding 30–40 turns without summarization will produce Context Rot. Tier 1 pruning is not optional at production scale.

- It cannot distinguish architecture failures from model failures without a structured diagnostic process. The four-step diagnostic sequence in the 'What This Argument Does Not Cover' section is the minimum process for attributing a production failure to the correct layer.

Unit Economics: The Total Cost of Inference

The TCI per resolved ticket is derived below for both a flat-context architecture and the tiered architecture. Figures use per-token pricing at a mid-tier LLM provider as of early 2026, averaged over a resolved ticket of 4–5 turns.

Cost Component	Flat-Context Bot	Tiered Architecture	Derivation
Input tokens per query	~50,000 (full corpus)	~3,400 (context + top-3 + system prompt)	Flat: full documentation loaded. Tiered: Tier 1 ~2k + Tier 5 top-3 ~900 + prompt ~500
Input cost per query	~$0.150	~$0.010	At $0.003 / 1k tokens
Output tokens per query	~300	~180	Average response length; targeted vs. verbose
Output cost per query	~$0.002	~$0.001	At $0.006 / 1k tokens
Retrieval cost per query	$0.000	~$0.001	Vector DB query at scale
Cost per resolved ticket (4–5 turns)	~$0.60–$0.76	~$0.05–$0.10	Flat: 4–5 × $0.152. Tiered: 4–5 × $0.012 + overhead
Cost per 10,000 tickets	~$7,000	~$1,000	Consistent with published $8,450 vs. $1,000 comparison

Table 8. TCI derivation. The 6–8x cost differential is caused by input token volume reduction, not model selection.

The quantitative conclusion: tiered retrieval reduces input token cost by approximately 93% compared to flat-context loading. This single architectural choice routing only relevant content to the attention window is responsible for nearly the entire cost differential. Model selection, prompt engineering, and output format optimization together account for the remaining 7%. The architecture is the leverage point.

Learning Outcomes and Evidence Structures

Each outcome below specifies the data the student generates, the analysis performed, and the artifact produced. A third party should be able to evaluate whether the outcome has been met by examining the artifact alone.

Level	Outcome	Evidence Structure
1. Remembering	List the five memory tiers and the storage technology for each	Annotated diagram produced from memory (no reference materials), compared against Table 3. Discrepancies documented in one sentence per discrepancy.
2. Understanding	Explain the Lost in the Middle effect and why it necessitates tiered retrieval over flat context loading; explain why Air Canada's failure was architectural, not a model hallucination	Written explanation (300 words) including: the positional mechanism, a numeric performance drop (from Liu et al. 2023 or equivalent), and one sentence connecting the mechanism to the Air Canada case.
3. Applying	Implement a HybridSearchRetriever combining vector similarity with lexical matching for policy queries	Working Python function passing four provided test cases: (a) natural-language query, (b) exact policy name query, (c) product SKU query, (d) ambiguous query. Test case results are the evidence.
4. Analyzing	Trace the causal chain from a small working memory buffer to the Amnesiac Pivot failure	Annotated conversation log from the demo notebook (max_history_turns = 3, refund scenario) identifying: (a) the turn at which the goal token was evicted, (b) the token count at that turn, (c) the turn at which agent behavior changed, (d) a 150-word written diagnosis connecting the eviction to the behavioral change.
5. Evaluating	Audit a support agent's deflection rate and diagnose whether failures are caused by retrieval failure (Tier 5) or goal loss (Tier 1)	Conversation logs from two notebook configurations: (a) standard, (b) Tier 5 disabled. Deflection rate and failure-mode distribution calculated for each. Written diagnosis with quantitative support. One alternative interpretation addressed and rebutted with evidence from the logs.
6. Creating	Design a memory architecture for a healthcare support agent under HIPAA constraints, and specify a Conflict Resolution Protocol for contradictory Tier 2 / Tier 5 outputs	Written architecture specification (one page) addressing: (a) which tiers are prohibited from retaining PHI beyond the treatment encounter, (b) which tiers are required for safe operation, (c) what the implementation changes from the My AskAI baseline, (d) one mandatory operational human decision node specific to the HIPAA context. Protocol specification: decision rule for Tier 2/5 conflict, human escalation condition, logging requirement, and one synthetic edge case that tests the non-obvious branch.

Table 9. Bloom's Taxonomy outcomes with evidence structures. Levels 4–6 require the demo notebook.

Triggerable Exercises

Both exercises are reproducible from a fresh clone of the repository. The failure in each case is observed, not described.

Exercise A: Trigger the Amnesiac Pivot

1. Set max_history_turns = 3 in the demo notebook.
2. Run the refund scenario: the simulated user reports a defective item and requests a refund.
3. Send three unrelated product questions as the next three turns.
4. Observe: the agent closes the session without completing the refund. Record the conversation log.
5. Add goal pinning: inject the active refund goal at the system prompt start every turn.
6. Rerun with identical input. Observe: the refund is completed.
7. Annotate the log from step 4: identify the turn at which the goal token was evicted (token count evidence required). This is the Level 4 evidence artifact.

Exercise B: Trigger the Episodic Blind Spot (The Priya Scenario)

1. Run a support session for a simulated user with a billing discrepancy. Mark it resolved.
2. Clear the episodic memory store only. Leave Tiers 1, 3, 4, and 5 intact.
3. Run a second session for the same user with the identical billing complaint.
4. Observe: the response is indistinguishable from the first session. No recurrence flag.
5. Restore the episodic store. Rerun the second session.
6. Observe: the agent flags the recurrence, surfaces the prior resolution, and recommends escalation.
7. Calculate deflection rate for each configuration across 10 simulated sessions. This calculation is the Level 5 evidence artifact.

Exercise C: Apply the Tetrahedron to a Novel Architecture

You are designing a memory architecture for a healthcare support agent serving patients of a large hospital system. HIPAA prohibits retention of Protected Health Information (PHI) beyond the treatment encounter unless specific consent conditions are met. Apply the Tetrahedron:

1. What does the HIPAA constraint require at the Structure level? Which tiers cannot retain PHI, and which must be designed to actively purge it?
2. What does it require at the Logic level? Which retrieval operations must be prohibited or modified?
3. What implementation choices does it eliminate? (Name at least two specific tools or configurations from the My AskAI baseline that cannot be used as specified.)

4. What Outcome trade-off does it force? What capability does the My AskAI baseline have that this architecture cannot have, and what is the measurable cost of that trade-off?

This exercise is the Level 6 evidence artifact. It requires applying the Tetrahedron as a generative tool to an architecture you did not design, under constraints that pull two of its four lenses in opposite directions.

The Architectural Claim, Demonstrated

Return to Jake Moffatt. He contacts Air Canada's support agent after a bereavement. He asks about bereavement fares. The agent tells him he can apply retroactively. He cannot. The agent fabricated a policy because there was no architectural constraint requiring it to retrieve and cite a source before generating a policy claim.

The fix is not a better model. The fix is a citation constraint at the Tier 5 retrieval layer: the agent is prohibited from generating policy-relevant content unless it can cite a retrieved source chunk. When no grounding source is available, it escalates. The legal liability of a confident wrong answer is replaced by the operational cost of an escalation a cost that is both smaller and recoverable.

The architecture is the leverage point. The model provides fluency. Memory provides context. The citation constraint provides accountability. Remove the constraint and fluency becomes a liability.

> Memory is not a feature. Memory is the system. The Air Canada agent had a model. What it did not have was a retrieval architecture that grounded that model's outputs in verifiable sources. $812.02 in damages is the measurable cost of an absent Tier 5 citation constraint.

References

- Liu, N. et al. (2023). Lost in the Middle: How Language Models Use Long Contexts. arXiv:2307.03172.
- Moffatt v. Air Canada, British Columbia Civil Resolution Tribunal, Case No. SC-CC-00462808, February 2024. [Tribunal ruling establishing airline liability for AI agent output.]
- Samsung bans employee use of AI like ChatGPT after data leak. Bloomberg / Reuters, May 2023. [Documents the three Samsung incidents and subsequent internal ban.]
- How My AskAI Built Self-Improving Support Agents. Qdrant Case Study, 2025. https://qdrant.tech/blog/case-study-my-askai/
- Agentic RAG vs Traditional RAG: Complete Guide. Mem0, 2025. https://mem0.ai/blog/agentic-rag-vs-traditional-rag-guide
- Building AI Agents That Actually Remember: A Deep Dive Into Memory Architectures. Towards AI, 2025.
- Context Engineering: LLM Evolution for Agentic AI. Elasticsearch Labs, 2025.
- The Unit Economics of AI Agents. Medium / Nikhil Sood, 2026.

- Characterizing Faults in Agentic AI: A Taxonomy of Types, Symptoms, and Root Causes. arXiv:2603.06847, 2026.